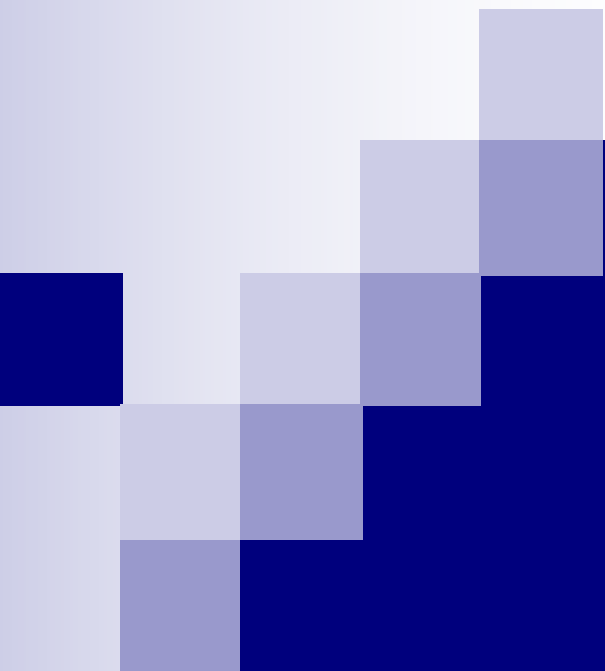




アルゴリズムとデータ 構造

第2回アルゴリズムと計算量

授業スライドURL:

<http://www-ikn.ist.nyu.edu/>

第1回イントロ(ガイダンス)

■ 今日の内容

- アルゴリズムとデータ構造とは何か
- 今日のアルゴリズム: 最大公約数問題を解く「ユークリッドの互除法」を学ぼう

■ ポイント

- アルゴリズムとデータ構造の定義
- 「ユークリッドの互除法」を理解すること
(情報理工学演習IIIを取っている人は初回の課題)

アルゴリズムとデータ構造とは？

アルゴリズム

- ◆ 入力データから正しい出力を計算するために，一連の手順を記述したもの．
- ◆ 正しさと効率が重要．

データ構造

- ◆ 計算のために，記憶領域に効率良くデータを格納するための配置法．
- ◆ 時間と記憶領域を効率化．

なぜ「アルゴリズムとデータ構造」を学ぶのか？

効率的なプログラムを作成できる
ようになるため

効率的とは

- 計算時間が短い
- メモリ使用量が少ない

メリット

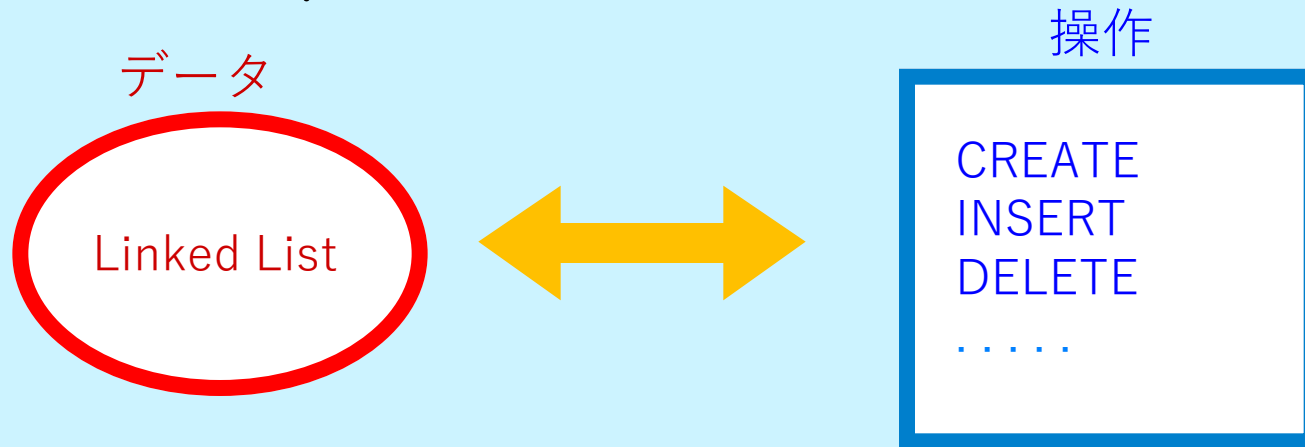
- アルゴリズムは**全てのプログラム言語**で役に立つ
- 情報理工学専攻大学院**入試に出題される科目**
- **IT関連に就職**する人には常識とされる

大事な考え方: 抽象データ型

抽象データ型 (Abstract Data Type)

データ型 を, それに適用される
に定めたもの.

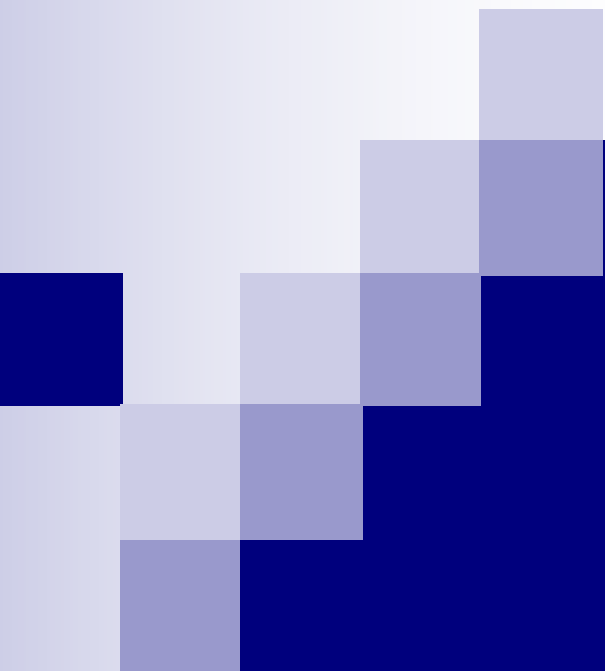
一組の操作 で抽象的



- データ構造には, 「どのように使うか?」と「それをどのように実現するか?」の二つの面がある.
- 抽象データ型の考え方は, とても重要である. 最近のほとんどのプログラム言語やライブラリーに取り入れられている. (例: C++, Java, Ruby, Python などなど)

データ構造とアルゴリズムの定番教科書

- この授業の教科書(一冊持っていると良い. 自習には本は必須.)
 - 茨木俊秀, Cによるアルゴリズムとデータ構造, 照晃堂, 2916円
 - 平田富夫, アルゴリズムとデータ構造<改訂 C言語版>, 森北出版, 2376円.
- Aho, Hopcroft, Ullman
 - The Design and Analysis of Computer Algorithms, Addison Wesley, 1974.(最初のデータ構造の教科書の一つ)
- Knuth
 - The Art of Computer Programming, Addison-Wesley
1巻(基本算法), 2巻(準数値算法), 3巻(整列と探索), 1975
 - 基本部分の百科辞典. 機械語で記述. 全部読んだ人は少ない?
- Cormen, Leiserson, Rivest, and Stein
 - Introduction to Algorithms, 3rd Edition, MIT Press, 2009.
 - 現在, 最も定評がある教科書. 企業やプロコンなどでの勉強会の定番. (日本語訳有, 3巻組. 1巻4000円くらい)



アルゴリズムとデータ 構造

第2回アルゴリズムと計算量

授業スライドURL:

<http://www-ikn.ist.hokudai.ac.jp/~yoshioka/algorithm/>

第2回アルゴリズムと計算量

■ 今日の内容

- アルゴリズムの時間計算量を理解する？
- 漸近的計算量「オーダー」(または O (ビッグオー))
- オーダーの計算の方法
- 例として、ユークリッドの互除法の時間計算量を求める

■ ポイント

- オーダー記法
- ビッグオー, ビッグオメガ, ビッグシータ
- 最悪時間計算量と平均時間計算量

凡例: 今日のスライドの背景の色分け



基本内容(白色): まずこれを理解しよう

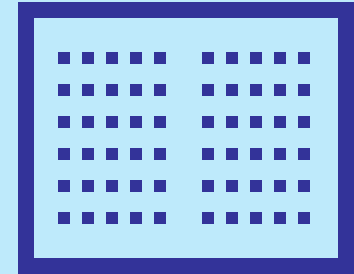


発展・議論・雑談(水色): 興味・余裕があれば, 読んでみよう

よいアルゴリズムとは？

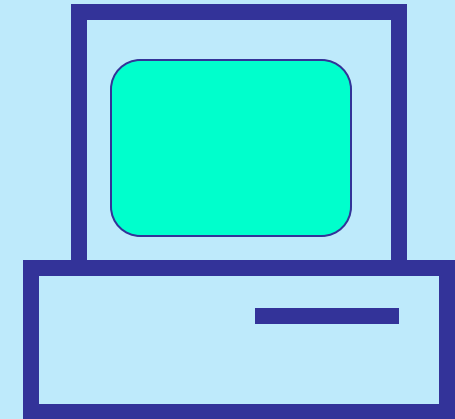
いろいろな性能

- 計算時間
- メモリサイズ
- 外部記憶への入出力数
- ネットワークの通信量



ほかの要素

- モジュラリティ Modularity
- 再利用可能性 Reusability
- 読みやすさ Readability
- 移植しやすさ Portability
- ...



ここでは、「性能」のみを測ることにする

アルゴリズムの性能の測り方

動機：アルゴリズム同士を比べて、性能の優劣をつけたい

ここでは、「計算時間」で比べることにする

👉 「計算時間」をどう測ればよいか？

計算時間の測り方

計算時間はいろいろな要素に依存する!

- ◆ 1. アルゴリズム
- ◆ 2. コーディングスタイル
- ◆ 3. 言語の種類とコンパイラの質
- ◆ 4. ハードウェアの性能
- ◆ 5. 入力そのもの

だから簡単には比べられ
しかし...

ここではアルゴリズムの違いだけに注目する!

そこで細かな部分は無視して
入力サイズに対する
計算時間の増加の
仕方だけを見よう.



計算量の評価

■ 計算量の種類

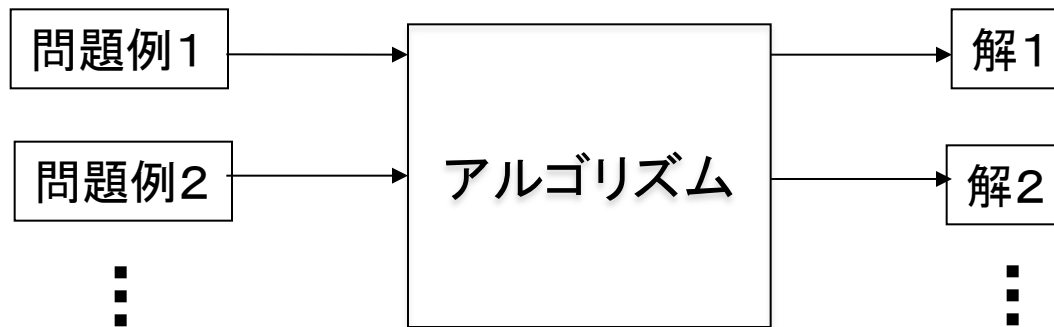
- 時間計算量(time complexity)
- 領域計算量(space complexity)

ステップ数

メモリ量

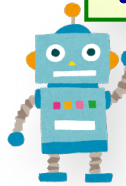
重要

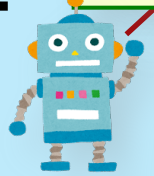
■ ある問題を解くアルゴリズム



1つの問題＝無限個の問題例の集合

考えて
みよう





考えてみよう: 時間計算量の測り方

- 基本的に、時間計算量は、入力を与えられたときのプログラム実行の**ステップ数**ではかる。
- すごく正確に、ステップ数をはかる必要があるか？
 - 例) ステップ数 $T(n) = 2n^2 + 5n + 1000$
- **ステップ数は、おなじアルゴリズムでも、プログラミング言語や、コンパイラ、書き方で変わる。**
- 入力 n に対して、だいたいの増加具合がわかればよいのではないか。
- → 漸近的計算量(オーダー)
 - 例) ステップ数 $T(n) = O(n^2)$

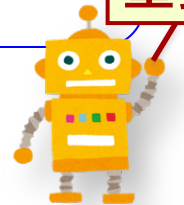
領域計算量にも同じ議論がなりたつ

計算量の評価法

問題例の規模によって計算量が変わる
⇒問題例の規模に応じた評価

入力長 n の関数 $T(n)$ として計算量を評価

重要



入力長および計算量の測り方は結構難しい...

疑問

- 入力長はどうやって測るのか？
- 計算時間は厳密に測るのか？

入力サイズの測り方の例

問題例 = 具体的な入力値のこと

考えて
みよう



問: それぞれの問題(例)で、
入力サイズは何になるか?

問題と問題例

GCD (Greatest Common Diviser) [最大公約数を求める問題]

問題

入力: 正整数 a_0, a_1

出力: a_0 と a_1 の最大公約数

👉 入力サイズ = a_0 と a_1 のビット長

問題例 $(a_0, a_1) = (28, 12), (987654321, 123456789)$

SUBSET-SUM [部分和问题]

問題

入力: $n+1$ 個の正整数 $a_0, a_1, \dots, a_{n-1}, b$

出力: $\sum_{j \in S} a_j = b$ となる $S \subseteq \{0, 1, \dots, n-1\}$ が存在したら yes、存在しなかったら no

👉 入力サイズ = 数の個数 $n+1$
(または全部の数のビット長の和)

問題例 $(a_0, a_1, a_2, b) = (1, 2, 3, 5), (a_0, a_1, a_2, a_3, a_4, a_5, b) = (1, 2, 4, 8, 16, 32, 60)$

入力サイズの測り方の例

問題と問題例

問題

EULER-PATH [オイラー路を求める問題]

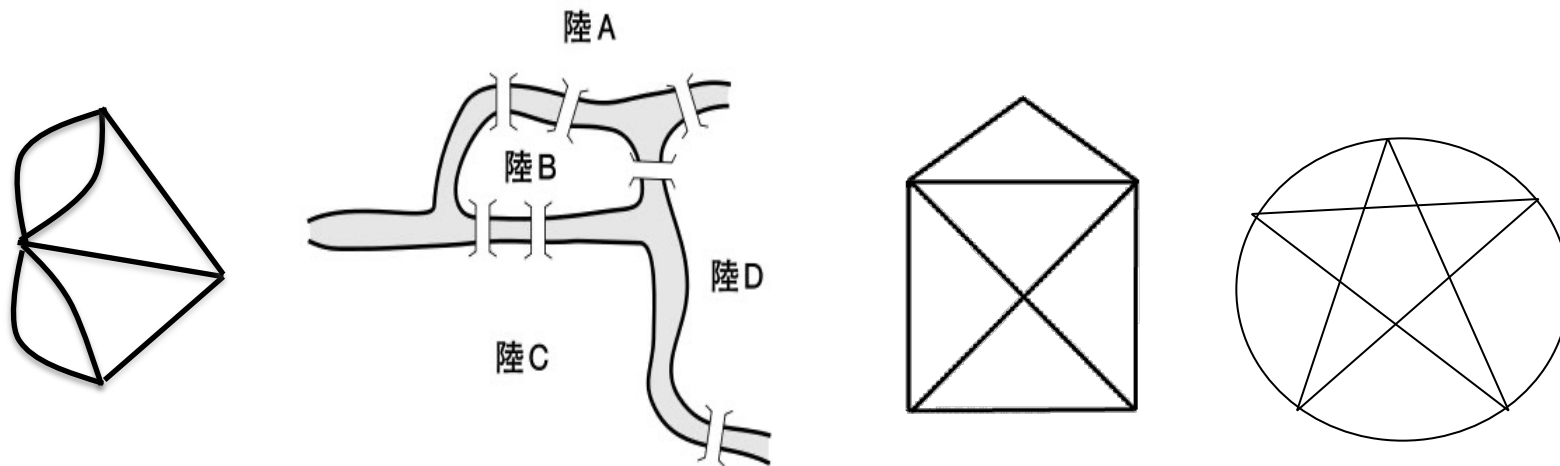
入力: グラフ $G = (V, E)$

出力: オイラー路 (各辺を1度だけ通る路) があれば、それらの路の1つ、なければ「なし」

👉 入力サイズ =
すべての頂点数と辺数の

和

問題例



$V = \{v_1, v_2, v_3, v_4\}$ ケーニッヒベルグの橋渡し問題
 $E = \{e_1, e_2, e_3, e_4, e_5, e_6\}$

例) 入力サイズ = 頂点数と辺数の和 = $4 + 6 = 10$

計算量の評価法

問題例の規模によって計算量が変わる

⇒問題例の規模に応じた評価

⇒**入力長 n の関数 $T(n)$ として計算量(計算時間)を評価**

重要



入力長および計算量は計算コストモデルに依存

■ 定数(一様)コストモデル

基本

すべての数を1語とみなし、どの基本命令も単位時間で実行できると仮定

■ 対数コストモデル

発展

各々の数はそれを表現するのに必要なビット数の語数が必要であり、基本命令の実行はビット数(表す値の対数)に比例した時間が必要と考える

大きな数字を扱うことが本質的な問題は**対数コストモデル**、そうでない場合は**定数コストモデル**

(例) GCDの入力(a_0, a_1)

SUBSET-SUMの入力 $a_0, a_1, \dots, a_{n-1}, b$

EULER-PATHの入力 V, E

対数コストモデル $\lceil \log a_0 \rceil + \lceil \log a_1 \rceil$

定数コストモデル $n+1$

定数コストモデル $|V| + |E|$
頂点の数と辺の数

実際には要素を分ける区切り文字が必要だがオーダー評価のため定数倍は影響ない

アルゴリズムとデータ構造

ここは大事！

では、計算量(計算時間)は
厳密にステップ数で測るの
か？

 No!

1.3 計算量の評価、漸近的計算量

<http://www.shutterstock.com/>

「オーダー」記法の勉強

「XXアルゴリズムの計算時間は入力の二乗時間オーダー」
「クイックソートアルゴリズムは $O(n \log n)$ 時間」
オーダーとは何か？

漸近的計算量(上界): O (ビッグオー)

漸近的上界 = asymptotic upper bound



「このアルゴリズムの計算時間は、大まかに言えば、これより小さいよ」ということ

オーダー評価

- 計算量 $T(n)$ は十分大きな n に対して評価する。
- 定数倍の差はないものとみなす。

単にオーダー記法ともいう

定義: 漸近的上界(ビッグオー記法) 記号 O

$T(n) = O(f(n)) \Leftrightarrow$ ある実数 $c > 0$ と自然数 n_0 が存在して、
全ての $n \geq n_0$ に対して、 $T(n) \leq cf(n)$ が成り立つ

「 $T(n)$ は、オーダー $f(n)$ である」または、「 $T(n)$ は、ビッグオー $f(n)$ である」と読む

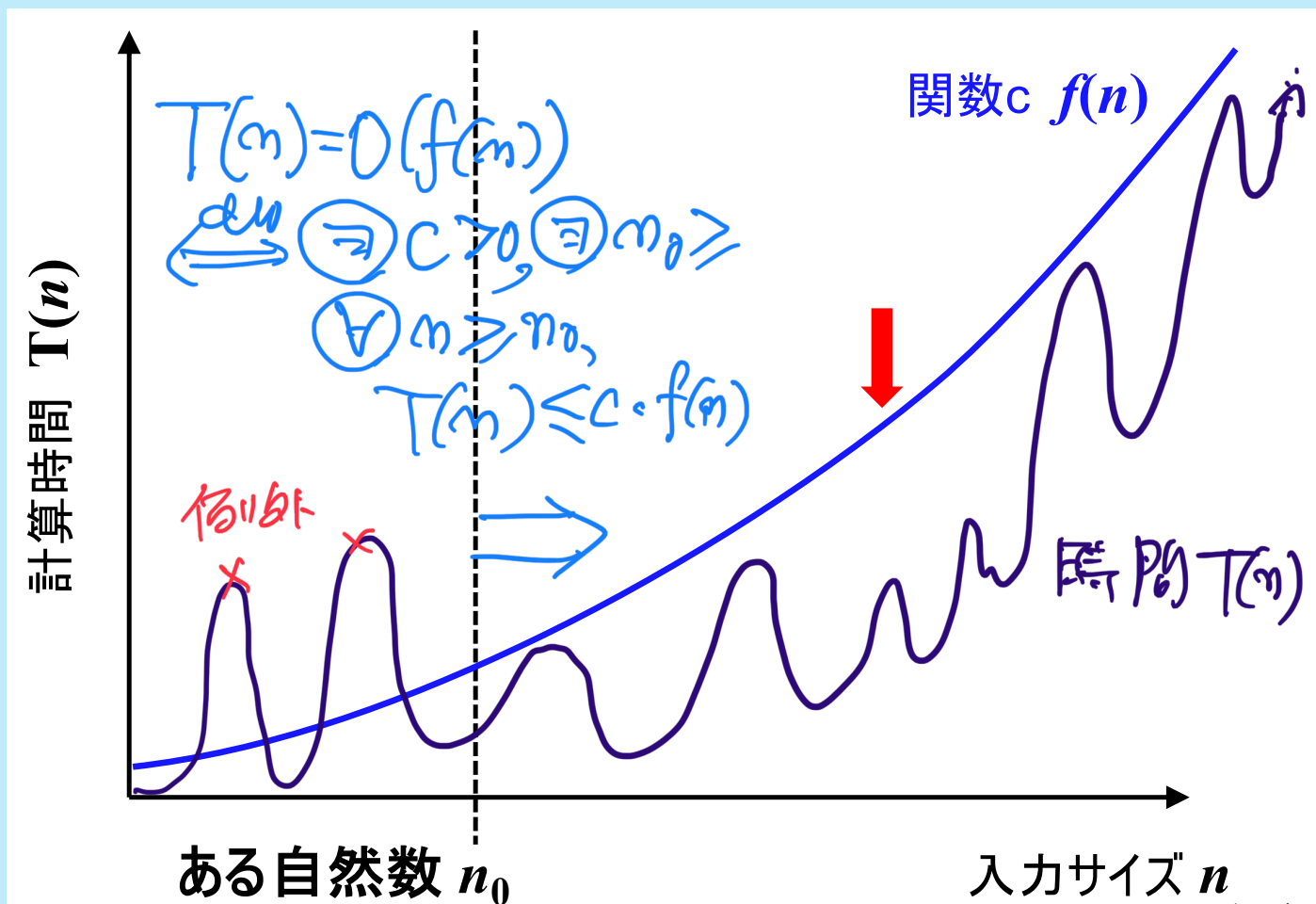
読み方「関数 $T(n)$ は $f(n)$ で上から抑えられる」

漸近的上界はいくらでも存在するができるだけ単純で精度のよいものがよい

- (1) $2n^2 + 5n + 1000 = O(n^2)$ ← 一番良い best!
- (2) $2n^2 + 5n + 1000 = O(n^2 + n)$ ← (1)より複雑
- (3) $2n^2 + 5n + 1000 = O(n^3)$ ← (1)より精度が悪い

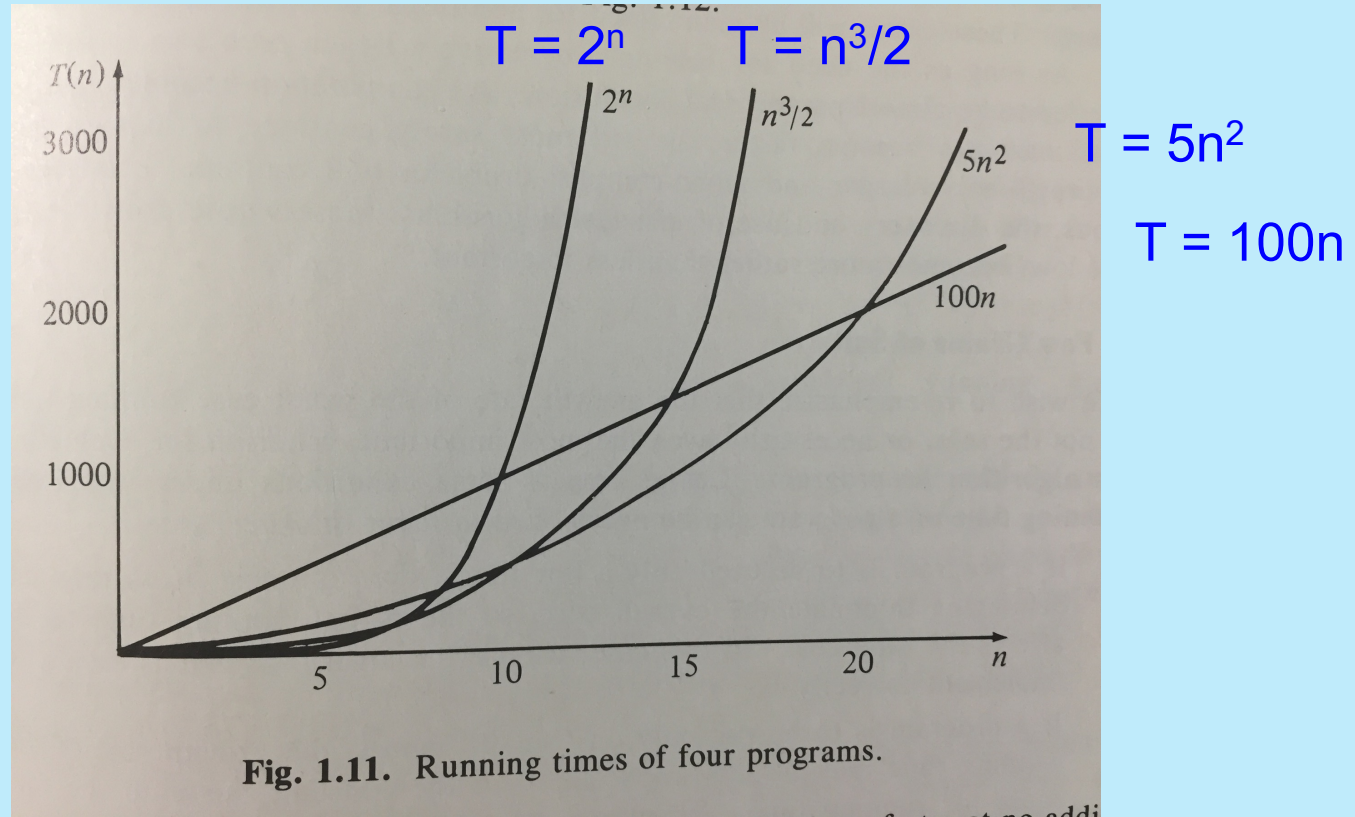
漸近的計算量: O (ビッグオー)

読み方「関数 $T(n)$ は $f(n)$ で上から抑えられる」



なぜ計算時間をオーダーで測るのか？

質問： 問題のサイズを大きくしていったらどうなるか？



- それぞれ, 2^n , $n^3/2$, $5n^2$, $100n$ の計算時間をもつ4つのプログラムに対して, その入力 n を増やしながら走らせたときの計算時間のグラフ.
- オーダーが大きいほど, 係数によらず計算時間が急速に増加する

なぜ計算時間をオーダーで測るのか？

- 質問： 時間をかけた分だけ大きなサイズの問題が解けるか？

smaller percentage increase in problem size. ... solve only small problems no matter how fast the underlying computer.

Running Time $T(n)$	Max. Problem Size for 10^3 sec.	Max. Problem Size for 10^4 sec.	Increase in Max. Problem Size
$100n$	10	100	10.0
$5n^2$	14	45	3.2
$n^3/2$	12	27	2.3
2^n	10	13	1.3

Fig. 1.12. Effect of a ten-fold speedup in computation time.

- 図) 4つプログラムに対して、左欄から、その計算時間と、それぞれ、1千秒と1万秒で解ける最大の問題サイズを示す。
- 観察： 計算時間を10倍にすると、 $O(n)$ 時間アルゴリズムなら10倍のサイズの問題がとける。しかし、 $O(n^3)$ 時間なら2.3倍、 $O(2^n)$ 時間なら1.3倍しか解ける問題のサイズが大きくなる！

漸近的計算量の性質

性質: $T_1(n)=O(f(n))$, $T_2(n)=O(g(n))$ のとき、次の等式が成立

$T_1(n)+T_2(n)=O(\max\{f(n),g(n)\})$ ← いくつかの処理を順次行う場合は一番遅い処理が全体の処理速度を支配する

$T_1(n)T_2(n)=O(f(n)g(n))$ ← 処理を繰り返し行くとその回数分時間がかかる

↑
証明してみよう!

(例) $n^2+n^3=O(n^3)$, $n^2 \cdot n^3=O(n^5)$

オーダー表記の注意点

左辺の精度 $\geq$ 右辺の精度

(例) ○ $2n^2+5n+1000 = O(n^2)$
 × $O(n^2)=2n^2+5n+1000$

漸近的計算量(下界) : Ω (ビッグオメガ)

「このアルゴリズムの計算時間は、これより大きいよ」ということ 漸近的下界=asymptotic lower bound

定義: 漸近的下界(ビッグオメガ記法) 記号 Ω

発展

$T(n) = \Omega(f(n)) \Leftrightarrow$ ある c, n_0 が存在して全ての $n \geq n_0$ に対して
 $T(n) \geq cf(n)$ が成り立つ

「 $T(n)$ はビッグオメガ $f(n)$ である」と読む

意味「関数 $T(n)$ は $f(n)$ と同じかもっと大きい」

読み方「関数 $T(n)$ は $f(n)$ で下から抑えられる」

(例) $2n^2 + 5n + 1000 = \Omega(n^2)$

$$T(n) = \begin{cases} n^2 & \text{if } n \text{ は奇数} \\ n^3 & \text{if } n \text{ は偶数} \end{cases} \Rightarrow T(n) = \Omega(n^2)$$

下の定義では
 $\Omega(n^3)$

以下の定義を Ω の定義として採用することもある。(研究論文ではこちらも多い)

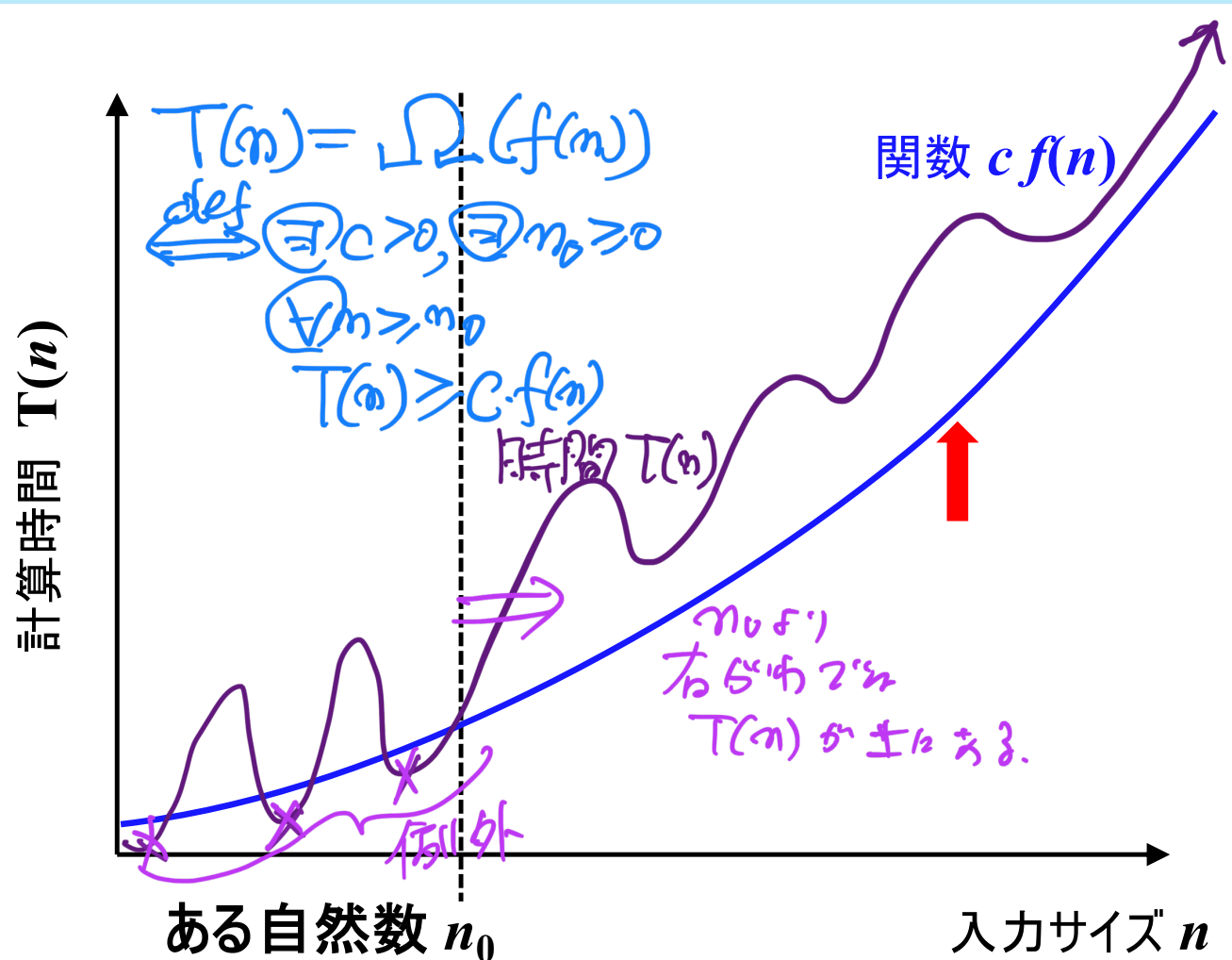
別定義: 漸近的下界(ビッグオメガ記法の別定義)

$T(n) = \Omega(f(n)) \Leftrightarrow$ ある c が存在して無限個の n に対して
 $T(n) \geq cf(n)$ が成り立つ

上の定義の方が厳しいので上の定義を満たせばこちらの定義も満たす

漸近的計算量: Ω (ビッグオメガ)

読み方「関数 $T(n)$ は $f(n)$ で下から抑えられる」



漸近的計算量: Θ (ビッグシータ)

定義: 漸近的にタイトな限界(asymptotic tight bound) 記号 Θ

発展

$T(n)=\Theta(f(n)) \Leftrightarrow$ ある実数 $c_0, c_1 > 0$ と自然数 n_0 が存在して全ての $n \geq n_0$ に対して $c_0 f(n) \leq T(n) \leq c_1 f(n)$ が成り立つ

「 $T(n)$ はビッグシータ $f(n)$ である」と読む

意味「関数 $T(n)$ は $f(n)$ とだいたい同じ」

読み方「関数 $T(n)$ は $f(n)$ で上と下から抑えられる」

(例) $2n^2 + 5n + 1000 = \Theta(n^2)$

$$T(n) = \begin{cases} n^2 & \text{if } n \text{ は奇数} \\ n^3 & \text{if } n \text{ は偶数} \end{cases} \Rightarrow T(n) \neq \Theta(n^3)$$

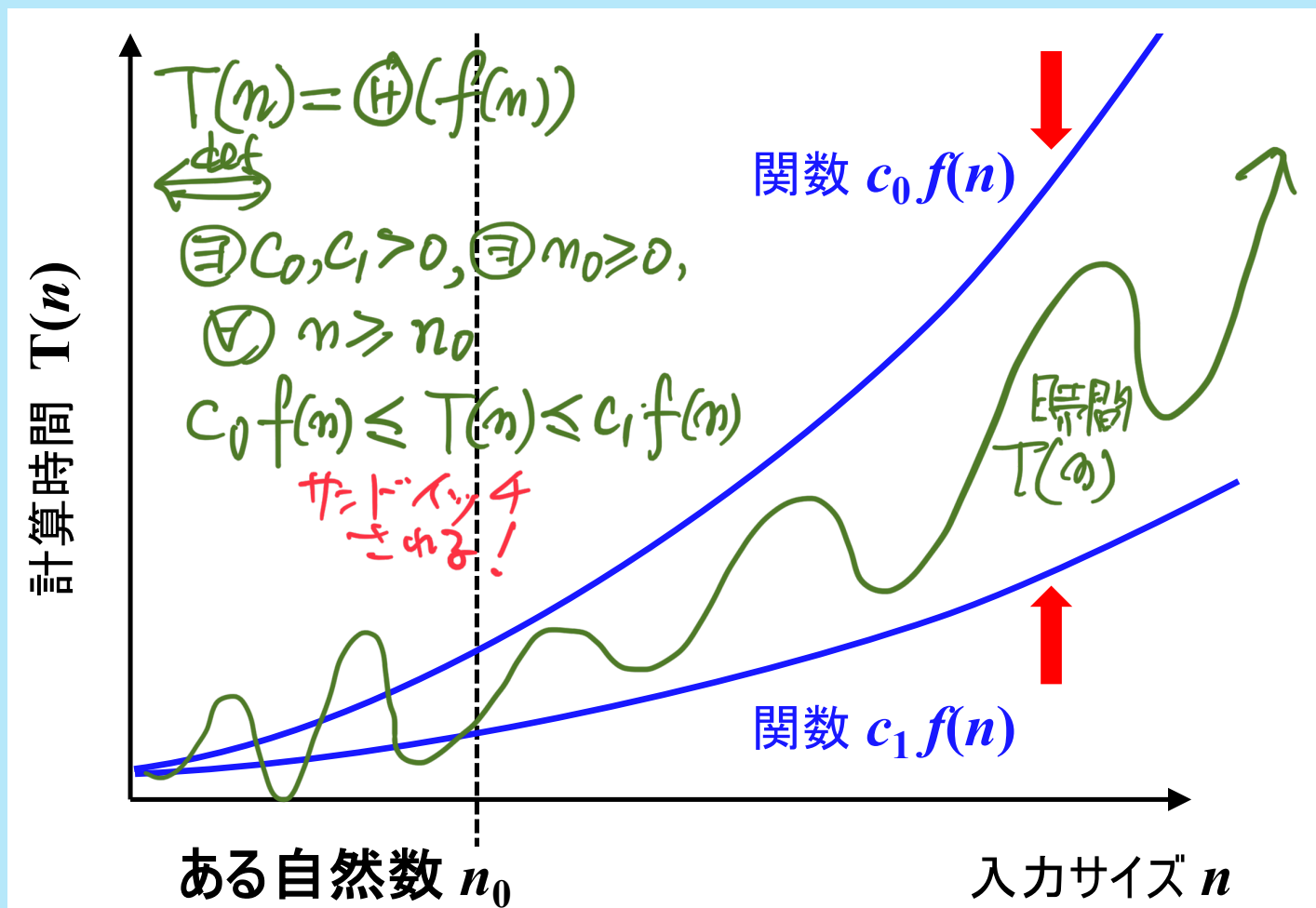
一般に

$$T(n) = \Theta(f(n)) \Leftrightarrow T(n) = O(f(n)), T(n) = \Omega(f(n))$$

Ω の別定義であれば $\Rightarrow$ のみ成立

漸近的計算量: Θ (ビッグシータ)

読み方「関数 $T(n)$ は $f(n)$ で上と下から抑えられる」



教養として o と ω も知っておこう！

$T(n)$ は $f(n)$ より真に小さいこと

漸近的にタイトでない上界

記号 o

発展

$T(n)=o(f(n)) \Leftrightarrow$ 任意の 定数 $c>0$ に対し,ある n_0 が存在して
全ての $n \geq n_0$ に対して $T(n) < cf(n)$ が成り立つ

「 $T(n)$ はスモールオー $f(n)$ である」

$T(n)$ は n が無限大に近づくにつれて $f(n)$ に対して相対的に小さくなる、
つまり、 $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$ となる。

(例) $2n = o(n^2)$, $2n^2 \neq o(n^2)$

$T(n)=o(f(x))$
 $\Updownarrow$
 $f(x)=\omega(T(n))$

漸近的にタイトでない下界

記号 ω

$T(n)$ は $f(n)$ より真に大きいこと

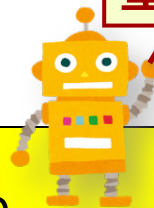
$T(n)=\omega(f(n)) \Leftrightarrow$ 任意の 定数 $c>0$ に対し,ある n_0 が存在して
全ての $n \geq n_0$ に対して $T(n) > cf(n)$ が成り立つ

「 $T(n)$ はスモールオメガ $f(n)$ である」

$T(n)$ は n が無限大に近づくにつれて $f(n)$ に対して相対的に大きくなる、
つまり、 $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = \infty$ となる。

(例) $2n^2 = \omega(n)$, $2n^2 \neq \omega(n^2)$

オーダーの計算法：基本のやりかた

重要

規則1: $T(n)$ が n の多項式ならば, 最大次数の項のオーダーになる

例: $2n^2 + 3n + 100 = O(n^2)$

例: $10n + 2\sqrt{n} + 5 = 10n^1 + 2n^{0.5} + 5 = O(n)$

規則2: 次のオーダーの式が成立する:

重要

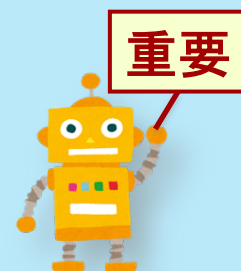
- $\log(n) = O(n)$
- $n = O(2^n)$
- 任意の $c > 0$ に対して, $\log n = O(n^c)$

重要

規則3: $T(n)$ がいくつかの項の和ならば, 最大次数の項のオーダーになる

例: $3n^2 + 2\sqrt{n} + 100 \log n + 5 = O(n^2)$

オーダーの計算法：自分で計算してみる 増加のオーダーによる比較(1/2)



計算時間が $T_1(n)$ と $T_2(n)$ のアルゴリズムではどちらが速いか？
⇒増加のオーダーが小さい方が速い

規則4: $T_1(n)$ よりも $T_2(n)$ の方が増加のオーダーが大きい

$$\Leftrightarrow \lim_{n \rightarrow \infty} \frac{T_1(n)}{T_2(n)} = 0 \quad \Leftrightarrow \lim_{n \rightarrow \infty} \frac{T_2(n)}{T_1(n)} = \infty$$

規則5: $T_1(n)$ と $T_2(n)$ は増加のオーダーが等しい

$$\Leftrightarrow T_1(n) = \Theta(T_2(n)) \quad \Leftrightarrow T_2(n) = \Theta(T_1(n))$$

$$T_1(n) \text{ と } T_2(n) \text{ は増加のオーダーが等しい} \Leftrightarrow \lim_{n \rightarrow \infty} \frac{T_1(n)}{T_2(n)} = c \text{ for some } 0 < c < \infty$$

オーダーの計算法: 増加のオーダーによる比較(2/2)

発展

規則6: l'Hospitalの法則

1. $f(x), g(x): \mathbb{R} \rightarrow \mathbb{R}$ が微分可能で $f(a)=g(a)=0$, $x=a$ 以外で $g'(x) \neq 0$ であれば

$$\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$$

2. $f(x), g(x): \mathbb{R} \rightarrow \mathbb{R}$ が微分可能で $\lim_{x \rightarrow \infty} f(x) = \lim_{x \rightarrow \infty} g(x) = \infty$ であれば

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \lim_{x \rightarrow \infty} \frac{f'(x)}{g'(x)}$$

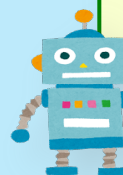
(例) $n^{0.01}$ と $\log^{100} n$ ではどちらが漸近的増加率が大きいのか?

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\log^{100} n}{n^{0.01}} &= \lim_{n \rightarrow \infty} \frac{100 \log^{99} n \times (1/n)}{0.01 n^{-0.99}} = \lim_{n \rightarrow \infty} \frac{100 \log^{99} n}{0.01 n^{0.01}} \\ &= \lim_{n \rightarrow \infty} \frac{100 \cdot 99 \log^{98} n}{0.01^2 n^{0.01}} = \dots = \lim_{n \rightarrow \infty} \frac{100!}{0.01^{100} n^{0.01}} = 0 \end{aligned}$$

演習(発展問題)

(5) つぎのオーダー表記を簡略化せよ。

(a) $O(n\log n + n^2) + O(n^{1.83}\log n)$



考えて
みよう

明らかに $n\log n < n^2$, $n\log n < n^{1.83}\log n$ である。

また、

$$\lim_{n \rightarrow \infty} \frac{n^{1.83}\log n}{n^2} = \lim_{n \rightarrow \infty} \frac{\log n}{n^{0.17}} = \lim_{n \rightarrow \infty} \frac{1/n}{0.17n^{-0.83}}$$

$$= \lim_{n \rightarrow \infty} \frac{1}{0.17n^{0.17}} = 0$$

よって十分大きな n に対して $n^{1.83}\log n < n^2$ であるから

$$O(n\log n + n^2) + O(n^{1.83}\log n) = O(n^2)$$

(b) $O(n^{\log n} + n^{100} + n^{30}\log n)$

$n^{30}\log n < n^{100}$ は明らか。

また十分大きな n に対して $n^{100} < n^{\log n}$ が成り立つ。

よって

$$O(n^{\log n} + n^{100} + n^{30}\log n) = O(n^{\log n})$$

(c) $O(n^3 \sin^2 n) O(2^n / \log n)$

$\sin^2 n \leq 1$ であるから

$$O(n^3 \sin^2 n) O(2^n / \log n)$$

$$= O(n^3 2^n / \log n)$$

任意の $a, b > 0$ に対し

$$\lim_{n \rightarrow \infty} \frac{\log^a n}{n^b} = 0$$

であるから十分大きな n に対しては、

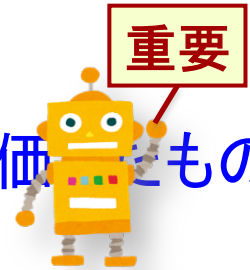
常に $\log^a n < n^b$ である。



慣れてきたら
この知識から
直接結論付けても
OK

アルゴリズムの計算量

計算量を問題例の入力長 N の関数としてオーダー評価したもの
以下の2つの評価法がある。



最悪計算量 (worst case complexity)

基本

入力長が N である問題例の中で最大の計算量

平均計算量 (average case complexity)

あまり使わない

入力長が N である問題例の計算量の期待値

(例) 入力長が N の問題例に対し確率 $(N-1)/N$ で $O(N)$ 、確率 $1/N$ で $O(N^2)$ であるようなアルゴリズムの計算量

最悪時間計算量 $O(N^2)$ 平均時間計算量 $O(N)$

通常は、最悪計算量で評価することが多い。

ここは大事！

実際に、最大公約数を求めるユークリッドの互除法アルゴリズムの時間計算量を求めてみよう

ユークリッドの互除法の時間計算量

GCDを解くのにどちらがよいアルゴリズム(1/2)?

考えて
みよう

入力: a_0, a_1 ($a_0 \geq a_1$)

入力長は $N = O(\log a_0)$
(a_0 を二進数で書いた時のビット数)



(a) ナイーブアルゴリズム

Step 1 $n \leftarrow a_1$

Step 2 n が a_0 と a_1 の約数であれば
 n を出力して停止

Step 3 $n \leftarrow n-1$ としてStep 2へ



(b) ユークリッドの互除法

Step 1 $a \leftarrow a_0, b \leftarrow a_1$

Step 2 $r \leftarrow a$ を b で割った余り

Step 3 $r=0$ ならば b を出力して停止

Step 4 $a \leftarrow b, b \leftarrow r$ としてStep 2へ

どっちが速い?

GCDを解くのにどちらがよいアルゴリズム(1/2)?

入力: a_0, a_1 ($a_0 \geq a_1$) 入力長は $N = O(\log a_0)$

(a) 単純なアルゴリズム

Step 1 $n \leftarrow a_1$

Step 2 n が a_0 と a_1 の約数であれば
 n を出力して停止

Step 3 $n \leftarrow n-1$ として Step 2 へ

考えて
みよう



ナイーブアルゴリズムの計算量

(1) 最悪の場合 ループの回数は $a_1 (\leq a_0)$
これは入力長 $N = \log(a_0)$ の指数

(2) ループ内において割り算が $O(\log^2 a_0)$
ステップ、その他の部分は $O(\log a_0)$
ステップ。全体では $O(a_0 \log^2 a_0)$ ステップ。

(3) したがって入力長を N とすれば
最悪時間計算量は $O(N^2 2^N)$ 。

指数時間アルゴリズム

GCDを解くのにどちらがよいアルゴリズム(2/2)?

入力: a_0, a_1 ($a_0 \geq a_1$) 入力長は $O(\log a_0)$

発展

(b) ユークリッドの互除法

Step 1 $a \leftarrow a_0, b \leftarrow a_1$

Step 2 $r \leftarrow a$ を b で割った余り

Step 3 $r=0$ ならば b を出力して停止

Step 4 $a \leftarrow b, b \leftarrow r$ として Step 2 へ

観察: 同様に、任意の i に対して、 $a_{i+2} < a_i/2$ 。つまり、ループを2回まわるごとに a の値が半分以下になる。

したがって、 $a_{k+1} = 0$ になるまで k 回ループが回ると、 $a_{2k} < a_0/2^k$ (1式)。

ここで右の計算より、**最悪の場合でも、ループの回数は $O(\log a_0)$ 回**。各ループ内において、割り算が $O(\log^2 a_0)$ ステップ、その他の部分は $O(\log a_0)$ ステップ。

よって全体では $O(\log^3 a_0)$ ステップ。

こちらがずっと速い!

ユークリッドの互除法の計算量

ループを回るごとに、次のような計算を順次して、 a の値が小さくなり、最後に $a_{k+1} = 0$ となる。

$$a_0 = q_1 a_1 + a_2,$$

$$a_1 = q_2 a_2 + a_3, \dots,$$

$$a_{k-1} = q_k a_k + a_{k+1}$$

$$a_0 - a_2 = q_1 a_1 \geq a_1 > a_2 \text{ であるから } a_2 < a_0/2.$$

ループ回数のオーダーの導出計算

$$0 = a_{2k} \leq a_0/2^k \quad ((1) \text{式より})$$

$$\rightarrow \log_2 0 \leq \log_2 a_0 - k \log_2 2$$

(両辺の $\log_2$ をとる)

$$\rightarrow k+1 \leq \log_2 a_0 \quad (\text{整理する})$$

$$\rightarrow k = O(\log_2 a_0) \quad (\text{オーダー})$$

考えてみよう



**多項式時間
アルゴリズム**

GCDの高速なアルゴリズム

- ユークリッドの互除法
- 知られている「最古のアルゴリズム」
- きわめて高速なアルゴリズム
 - 素朴なアルゴリズムより「指数的に高速」である(次回説明)
- 最大公約数に関する次の性質を利用する

プログラム設計道場 秘伝その2:
問題(解)の性質を利用しよう!
定義通りに解くだけでなく
(答えの「もう一つの顔」をつかもう)

性質: 非負整数 a と b ($a < b$) の最大公約数 $\text{GCD}(a, b)$ について次が成立する

- $\text{GCD}(0, b) = b$
- $\text{GCD}(a, b) = \text{GCD}((b \% a), a)$.

補足

- aとbのGCD は, 大きい方bを小さい方 aで割った剰余(余り, mod, remainder) $r = (b \% a)$ とaのGCDと等しい.
- これを繰り返すごとに, 正数 a がより小さい正数 r で置き換えられるので, だんだん小さくなって0に近づく

第2回アルゴリズムと計算量

■ 今日の内容

- アルゴリズムの計算量評価
- 漸近的計算量(いわゆるオーダー)
 - オーダーの計算の方法
 - 最悪計算量と平均計算量
- 発展: GCDアルゴリズムの時間計算量の計算
 - 解析により, GCDアルゴリズムは, ナイーブアルゴリズムより, はるかに高速なことがわかった

■ ポイント

- オーダー記法
- ビッグオー, ビッグオメガ, ビッグシータ